NumPy Cheatsheet

Core concepts · gotchas · quick reference

1. IMPORT

```
import numpy as np
```

2. CREATING ARRAYS

```
np.array([1, 2, 3]) # 1D array
np.array([[1,2],[3,4]]) # 2D array (matrix)
np.zeros((3, 4)) # 3x4 matrix of 0.0
np.ones((2, 3)) # 2x3 matrix of 1.0
np.full((2, 2), 7) # 2x2 filled with 7
np.arange(0, 10, 2) # [0, 2, 4, 6, 8]
np.linspace(0, 1, 5) # 5 evenly spaced values
np.eye(3) # 3x3 identity matrix
np.random.randint(0,10,(3,3)) # random ints 0-9
np.random.randn(3, 3) # normal dist, mean=0
np.random.seed(42) # reproducibility
```

```
random.random() needs a TUPLE for shape!
np.random.random(6, 1) # TypeError
np.random.random((6, 1)) # correct
```

3. SHAPE, SIZE & DIMENSIONS

```
a.shape # (2, 3)
a.ndim # 2
a.size # 6
a.dtype # int64
a.reshape(3, 2) # element count must stay same
a.reshape(-1) # flatten to 1D
a.flatten() # always returns a copy
# reshape(-1, 1) -> column vector
# reshape(1, -1) -> row vector
```

4. INDEXING & SLICING

```
a[0, 1] # single element (row 0, col 1)
a[2, -1] # last element of last row
a[0, :] # entire first row
a[:, 1] # entire second column
a[0:2, 1:3] # submatrix
a[a > 30] # boolean indexing - filter elements
```

```
Slices return VIEWS, not copies - edits affect the original!
b = a[0, :] # dangerous - view!
b = a[0, :].copy() # use .copy() to be safe
```

5. MATH OPERATIONS

```
a + b a - b a / b a ** 2 # element-wise
a + 10 # scalar broadcast
np.dot(a, b) # matrix multiply
a @ b # same, cleaner
np.sum(a) # sum all elements
np.sum(a, axis=0) # sum along columns
np.sum(a, axis=1) # sum along rows
np.mean(a) / np.max(a) / np.min(a)
np.argmax(a) # index of max
np.sqrt(a) / np.abs(a) / np.exp(a) / np.log(a)
```

```
* is element-wise, NOT matrix multiply!
~~a * b # element-wise only~~
a @ b or np.dot(a, b) # matrix multiply
```

6. BROADCASTING

```
# Dims compared from the RIGHT. Size-1 stretches to match.
a = np.array([[1,2,3],[4,5,6]]) # shape (2, 3)
b = np.array([10, 20, 30]) # shape (3,) -> (2, 3)
a + b # [[11,22,33],[14,25,36]]
c = np.array([[100],[200]]) # shape (2, 1) -> (2, 3)
a + c # [[101,102,103],[204,205,206]]
```

```
(3,) and (3,1) are NOT the same! (3,) = 1D | (3,1) = 2D column vector
```

7. MATRIX OPERATIONS

```
a.T # transpose
np.linalg.inv(a) # matrix inverse (square only)
np.linalg.det(a) # determinant
np.linalg.eig(a) # eigenvalues & eigenvectors
np.vstack([a, b]) # stack vertically (add rows)
np.hstack([a, b]) # stack horizontally (add cols)
```

8. DATA TYPES (DTYPE)

```
np.array([1,2,3], dtype=np.float32)
# int8/16/32/64 float16/32/64 bool complex64
np.array([1.7, 2.9]).astype(int) # [1, 2] TRUNCATES!
np.array([1, 2, 3]).dtype # int64
np.array([1.0, 2, 3]).dtype # float64 (one float -> all float)
```

```
astype(int) TRUNCATES, does not round!
~~a.astype(int) # [1.9, 2.9] -> [1, 2]~~
np.round(a).astype(int) # round first, then convert
```

9. USEFUL UTILITY FUNCTIONS

```
np.isnan(a) # boolean mask for NaN
np.isinf(a) # boolean mask for Inf
np.unique(a) # sorted unique values
np.sort(a) # sorted copy
np.where(a > 2, 1, 0) # conditional: 1 if a>2 else 0
a.copy() # explicit copy (avoid view bugs)
np.save('arr.npy', a) # save array
np.load('arr.npy') # load it back
np.savetxt('f.csv', a, delimiter=',')
```

10. QUICK GOTCHAS REFERENCE

WRONG	RIGHT
`np.random.random(3, 3)`	`np.random.random((3, 3))`
`a * b  (matrix multiply)`	`a @ b  or  np.dot(a, b)`
`b = a[0,:]  then edit b`	`b = a[0,:].copy()`
`astype(int) on [1.9, 2.9]`	`np.round(a).astype(int)`
`reshape(2,2) on 3 elements`	`element count must match`